arXiv:2606.29193v1 [cs.SE] 28 Jun 2026

A Multi-Dataset Benchmark for Evaluating LLM Agents in Microservice Failure Diagnosis

Yuanhong Cai
Computer Network Information Center, Chinese Academy of Sciences
Beijing, China

Xiaohui Nie
Computer Network Information Center, Chinese Academy of Sciences
Beijing, China

Kanglin Yin
Key Laboratory for Satellite Digitalization Technology, Chinese Academy of Sciences
Shanghai, China

Changhua Pei
Computer Network Information Center, Chinese Academy of Sciences
Beijing, China

Yongqian Sun
Nankai University
Tianjin, China

Shenglin Zhang
Nankai University
Tianjin, China

Haibin Liu
Alibaba Cloud Computing Company
Hangzhou, China

Guiyang Liu
Alibaba Cloud Computing Company
Hangzhou, China

Xidao Wen
Alibaba Cloud Computing Company
Hangzhou, China

Fang Situ
Alibaba Cloud Computing Company
Hangzhou, China

Dan Pei
Tsinghua University
Beijing, China

Abstract

LLM-based agents are reshaping microservice operations into *AgentOps*, where benchmarks are key to evaluating failure diagnosis over multimodal observability data. However, existing benchmarks remain largely outcome-oriented: they score only the final answer and fail to assess the systematic reasoning process in failure diagnosis. We address this gap by introducing two large-scale datasets (***AIOps2025*** and ***RCA100***) under a reasoning-process evaluation paradigm that assesses agentic diagnostic capability along three dimensions: *Localization*—where the fault occurs, *Identification*—what type of fault it is, and *Reason*—whether the reasoning trace is grounded in relevant evidence. Together, the two datasets comprise over 500 expert-labeled failure cases across two representative microservice systems (HipsterShop and the OpenTelemetry Demo Store). They cover diverse fault scenarios across resource, network, runtime, middleware/database, and application-logic categories and provide fine-grained causal evidence to support agent learning and reasoning-process evaluation. Beyond scale and coverage, the datasets have been carefully labelled by domain experts and validated through large-scale competitions, supporting more than 6,000 participating teams. This makes them not only expert-labeled diagnostic datasets, but also competition-validated benchmarks for evaluating agentic failure diagnosis in real-world microservice environments. Datasets are available at https://www.aiops.cn/gitlab/aiops-live-benchmark/agenticopseval.

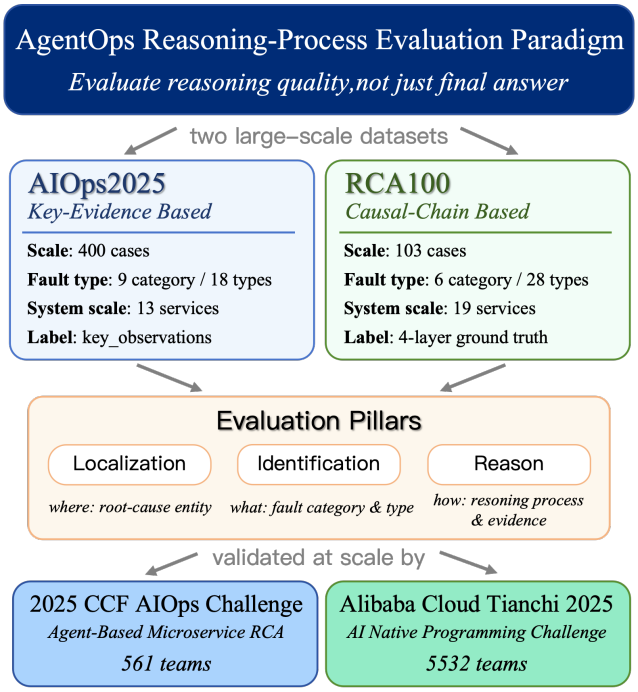

Figure 1: Overview of our benchmark.

1 Introduction

Microservice reliability is a fundamental concern in modern software operations [1, 27]. With chain-of-thought prompting [22] and ReAct-style tool use [20, 25], LLM-based AIOps agents have shown growing capability in diagnosing failures from multimodal observability data, including metrics, logs, and traces [9, 14, 19]. These advances have shaped the emerging *AgentOps* paradigm, spanning single-agent tool augmentation [4, 29] and multi-agent collaboration [16, 21, 28].

Evaluating agentic diagnostic capability is essential for assessing whether agent systems are practically usable in microservice operations. However, existing benchmarks for root cause analysis [18] and LLM-based agents [13] mostly focus on final-answer matching, ignoring how the diagnosis is derived. Such outcome-only evaluation can misjudge agent capability, since a correct answer may result from keyword matching rather than systematic, evidence-grounded reasoning. A more reliable benchmark should therefore evaluate not only root-cause accuracy, but also whether the agent can localize the fault, identify its root cause, and justify the diagnosis with relevant evidence or causal chain. Constructing such a reasoning-process benchmark raises two key challenges:

- **Insufficient reasoning-process evaluation.** Existing benchmarks typically label only the final root cause, making it difficult to distinguish evidence-grounded diagnosis from accidental keyword matching. A fair benchmark should further specify *which evidence is diagnostically relevant* and *how such evidence supports the causal reasoning process*.
- **Lack of large-scale validation.** Existing datasets are typically tested by only small expert groups. A robust benchmark should also be validated by large-scale users to ensure reliable evaluation across diverse agents.

To address these challenges, we present two large-scale microservice AIOps datasets, validated through two major national-level public competitions in 2025, for evaluating agentic anomaly detection, fault localization, and reasoning processes (Fig. 1). We organize the evaluation around three shared pillars: *Localization*—which entity causes the failure, *Identification*—what fault type occurs, and *Reason*—whether the agent grounds its reasoning in relevant evidence. ***AIOps2025*** evaluates key-evidence coverage by labelling per-modality key observations for open-ended anomaly descriptions in a self-hosted HipsterShop system. ***RCA100*** evaluates causal-chain coverage by providing structured alert inputs and four-layer ground truth—fault category, root-cause entity, causal propagation chain, and evidence checkpoints—on the OpenTelemetry Demo Store deployed on Alibaba Cloud ACK. In summary, the main contributions are as follows.

- **A reasoning-process evaluation paradigm.** We argue that RCA agent evaluation should move beyond final-answer or component-only scoring and assess whether an agent's reasoning trace is grounded in the right diagnostic evidence. We operationalize this idea through three shared pillars—*Localization*, *Identification*, and *Reason*—instantiated by two complementary labelling forms: *key-evidence* and *causal-chain*.
- **Two large-scale multimodal datasets.** We release 503 expert-labeled failure cases with ≈ 15.3 GB of multimodal observability data from two heterogeneous microservice architectures. Each dataset is paired with a deterministic scoring protocol aligned with the three pillars, turning reasoning-process evaluation into an executable benchmark.
- **Large-scale real-world competition validation.** The datasets powered two national-level public competitions: the *2025 CCF AIOps Challenge*[1] (561 teams) and *the Alibaba Cloud Tianchi 2025 AI-Native Programming Challenge*[2] (5,532 teams), involving 6,093 teams in total. To the best of our knowledge, this is the largest competition-validated dataset for microservice reasoning-process evaluation to date. The datasets are publicly available at https://www.aiops.cn/gitlab/aiops-live-benchmark/agenticopseval.

The remainder of this paper is organized as follows. Section 2 reviews related work, and Section 3 presents our design principles. Sections 4 and 5 introduce the two datasets, including their statistical properties and large-scale real-world validation. Section 6 discusses lessons, limitations, and broader uses, before Section 7 concludes.

2 Related Work

Existing failure-diagnosis datasets fall into three categories: those for traditional RCA methods, those for LLM-based agents, and interactive environments for agent training. Table 1 contrasts them with our work on case scale, modality coverage, reasoning-process labelling, and large-scale validation.

Datasets for traditional RCA. RCAEval [18] aggregates 735 cases across three open-source microservice systems with component-level root-cause labels, serving as the de-facto baseline for causal-graph and change-point methods (e.g., MicroScope [12], MicroRCA [23], RCD [8], BARO [17], DiagFusion [26]). The labels record *what* the answer is, not *how* a reasoner should reach it, leaving evidence-grounded diagnosis indistinguishable from keyword luck.

Datasets for LLM-based agents. OpenRCA [24] is the representative offline benchmark, asking an LLM to output a ⟨time, component, reason⟩ triple over 335 cases from three enterprise systems. It still grades the final answer only. Our two datasets enter this category and add explicit reasoning-process supervision: per-modality key evidence (*AIOps2025*) and a causal propagation chain with 661 evidence checkpoints (*RCA100*), so scores reflect diagnostic process quality.

Interactive environments for agent training. AIOpsLab [3] (88 tasks), ITBench [10] (94 scenarios), and SREGym [5] (114 problems) put agents inside live systems and score end-to-end execution. Evaluation is dominantly pass/fail or efficiency-only; only ITBench partially labels the reasoning process. These environments are complementary deployment targets, not substitutes: our datasets supply the missing process-level supervision and the first large-scale competition validation.

3 Design Principles

A useful RCA benchmark for LLM agents must control three axes: input-signal richness, fault-coverage breadth, and observability of the reasoning process. We adopt one design principle along each.

3.1 Multimodal Coverage

Diagnostic evidence is distributed across modalities with complementary roles: metrics expose quantitative trends, logs carry semantic error context, and traces record propagation along the call topology. None is individually sufficient—similar metric curves

[1] https://challenge.aiops.cn/home/competition/1920410697896845344

[2] https://tianchi.aliyun.com/competition/entrance/532387

Table 1: Existing AIOps benchmarks vs. our work on case scale, modality coverage, reasoning-process labelling, and large-scale validation. M / L / T / E / A / Topo = Metrics / Logs / Traces / Events / Alerts / Topology. ✓ = supported, × = not supported. "Partial" for ITBench means the ground truth includes the fault propagation chain and remediation steps, but not the per-step diagnostic evidence checkpoints.

Benchmark	#Cases	Modalities	Reasoning process label	Large-scale validation
RCAEval [18]	735	M+L+T	×	×
OpenRCA [24]	335	M+L+T	×	×
AIOpsLab [3]	88	M+L+T	×	×
ITBench [10]	94	M+L+T	✓ (partial)	×
SREGym [5]	114	M+L+T	×	×
***AIOps2025* (ours)**	**400**	M+L+T	✓ (key-evidence)	✓ (CCF / 561 teams)
***RCA100* (ours)**	**103**	M+L+T+E+A+Topo	✓ (causal chain)	✓ (Tianchi / 5,532 teams)

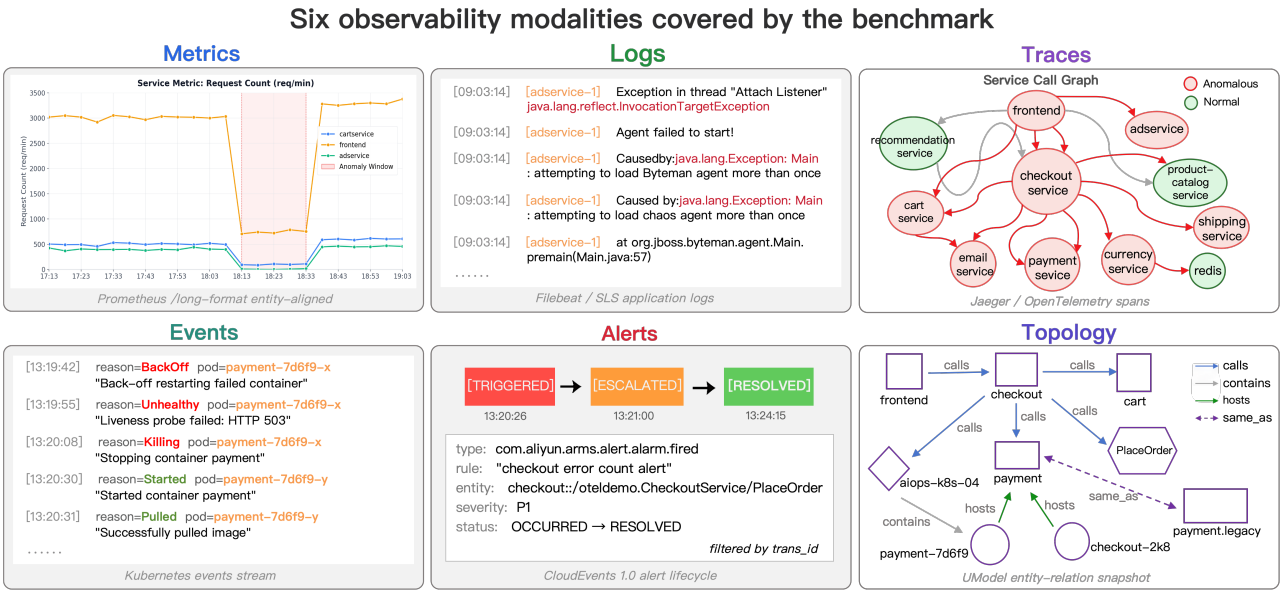

Figure 2: Multimodal observability data across the two datasets. The top three modalities (Metrics, Logs, Traces) constitute the base signal stack shared by *AIOps2025* and *RCA100*; the bottom three (Events, Alerts, Topology) are higher-order signals unique to *RCA100*. Anomalous behavior surfaces consistently across modalities via shared entity identifiers, letting an agent triangulate evidence along service / pod / entity dimensions.

stem from disparate causes, log errors may never be written before a container chokes, and JVM-internal anomalies are invisible to cross-service spans. A high-quality benchmark must therefore *force* cross-modal fusion rather than allow shortcuts through any single modality.

We therefore require **multimodal coverage**. *AIOps2025* provides the three base modalities; *RCA100* further adds Events, Alerts, and Topology (Fig. 2).

3.2 Hierarchical Fault Coverage

Microservice faults differ systematically in observability footprint, propagation scope, and reasoning difficulty depending on *where* and *what* they are. Restricting a benchmark to one entity layer or one fault family biases evaluation toward methods tuned to that slice and obscures cross-layer discrimination. We therefore require **hierarchical fault coverage** along two orthogonal axes.

Entity layer. *Service-level* faults affect every pod under a service (e.g., network attacks, erroneous deploys); *pod-level* faults touch a single instance and demand discrimination among replicas (e.g., pod failure / kill, I/O pressure); *node-level* faults span all services on the same host (e.g., node CPU / memory / disk pressure).

Fault category. The benchmark covers typical production fault families: *resource* (CPU, memory, disk, I/O), *network* (latency, loss, DNS), *runtime* (JVM CPU / GC / exception / latency), *middleware & database* (TiDB and Redis disturbances), and *application-logic* (erroneous deploy, traffic surge, rate limiting, null-pointer). Both datasets are designed under this principle.

3.3 Reasoning-Process Label

Traditional RCA benchmarks label only the final root-cause component—adequate for statistical and causal-graph methods, but insufficient under the LLM-agent regime, as it cannot separate evidence-grounded reasoning from keyword luck. From a fault perspective, a real failure is a propagation path along the call topology, not a single component; from an agent perspective, an LLM diagnoses step by step, not in one shot. A reasoning-process label must therefore expose both the fault's propagation structure and the per-step evidence the agent should consult along the way.

We therefore require **reasoning-process labels** that record, beyond the final root cause, the propagation structure of the fault and the evidence supporting each step of the diagnostic trace.

4 AIOps2025: 2025 CCF AIOps Challenge Dataset

AIOps2025 is built for the 2025 CCF AIOps Challenge and contains 400 fault cases drawn from a single distribution. The input is an open-ended natural-language anomaly description with a time window—mimicking free-text alert triage; the agent must perform data loading, anomaly detection, and cross-modal reasoning by itself, and is scored not only on the predicted root-cause component but also on the per-modality key evidence its reasoning trace covers.

4.1 System and Fault Injection

System. The underlying system spans three tiers (Fig. 3): the *client entry* tier (*frontend* gateway plus a load-generator), the *business microservice* tier built on **HipsterShop** (Google Cloud's open-source 10-microservice e-commerce demo) [6], and the *storage* tier with **TiDB** [7] (PD, TiKV, TiFlash) as the distributed SQL backend and a **Redis** cache. The whole stack is orchestrated by Kubernetes across 8 virtual machines, totalling 13 **services and** 33 **pods** (10×3 HipsterShop pods plus three TiDB components, 1 pod each). HipsterShop's call graph exercises synchronous HTTP/gRPC, asynchronous messaging, and cross-language services. **Differing from prior AIOps datasets that build on bare HipsterShop and confine fault scope to the application layer, *AIOps2025* also injects faults onto the storage tier**—resource, I/O, and network disturbances on TiDB components (PD, TiKV, TiFlash) and the Redis cache pod—so the benchmark exercises diagnostic reasoning across the application → storage propagation path that production incidents traverse but pure microservice demos systematically omit.

Fault injection. Faults are injected via **Chaos-Mesh** [2] at three hierarchical entity levels in line with Section 3.2: service, pod, and node. The dataset spans 9 fault categories and 18 fault types covering resource, network, runtime, application-logic, and infrastructure failures; Table 2 summarizes the categories with per-type case counts. Per-category injection parameters (durations, magnitudes, target-instance selection) are randomized within fixed ranges, released alongside the dataset.

4.2 Multimodal Data

Each case ships its full Metrics, Logs, and Traces over the anomaly window. Metrics come from Prometheus [19] (service/pod APM) plus node and TiDB-cluster infrastructure feeds; Logs from Filebeat; Traces from Jaeger [9]. The three modalities share service and pod names, anchoring an event to consistent identifiers across modalities; samples are shown in Fig. 2 (top). The dataset comprises 2,835 **Parquet files,** ≈ 269 **M rows, and** 11.9 **GB** over 18 calendar days, partitioned by phase / calendar day / modality and sliced hourly so participants can load only the relevant range without touching the full corpus.

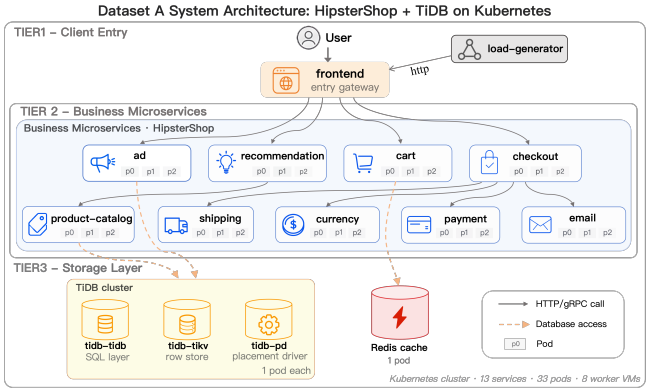

Figure 3: *AIOps2025* system architecture. Three tiers (client entry / business microservices / storage) on Kubernetes: 10 **HipsterShop services with** 3 **pods each, three TiDB components, and a Redis cache, deployed across** 8 **worker VMs.**

Table 2: *AIOps2025* fault taxonomy: 9 **categories** × 18 **fault types, with per-type case counts.**

Category	Fault type	Description	#Cases
node fault	node cpu stress	Node-level CPU saturation	23
	node memory stress	Node-level memory saturation	38
	node disk fill	Node-level disk exhaustion	21
network attack	network delay	Inter-service latency injection	25
	network loss	Packet loss between services	21
	network corrupt	Packet corruption between services	27
pod fault	pod failure	Whole-service or single-pod down	45
	pod kill	One-shot pod restart	15
jvm fault	jvm cpu	JVM CPU spike	13
	jvm gc	Stop-the-world GC pause	14
	jvm exception	Java exception throwing	13
	jvm latency	JVM-internal latency injection	15
stress test	cpu stress	Service-/pod-level CPU stress	22
	memory stress	Service-/pod-level memory stress	20
io fault	io fault	Disk I/O delay or error	28
erroneous change	code error	Buggy image deployed to a service	21
dns fault	dns error	DNS resolution failure	21
misconfiguration	target port misconfig	Service targetPort mis-binding	18
Total	**18 types**		**400**

```
{
  "uuid": "345fbe93-80",
  "fault_category": "stress test",
  "fault_type":     "cpu stress",
  "instance_type":  "service",
  "service":  "emailservice",
  "instance": "emailservice",
  "start_time": "2025-06-05T16:10:02Z",
  "end_time":   "2025-06-05T16:31:02Z",

  "key_observations": [
    {"type": "metric", "keyword": ["pod_processes"]},
    {"type": "metric", "keyword": ["rrt",
        "rrt_max"]},
    {"type": "metric", "keyword": ["pod_cpu_usage"]}
  ],
  "key_metrics": ["pod_cpu_usage"],
  "fault_description": [
    "high CPU utilization", "CPU saturation",
    "CPU overload", "CPU usage spike", "cpu stress"
  ]
}
```

Figure 4: *AIOps2025* ground-truth example: a service-level *cpu stress* case on *emailservice*. Per-modality *key_observations* and *key_metrics* drive the Explainability score; *fault_description* drives the Type Accuracy score.

4.3 Groundtruth Labeling

To ensure label reliability, each case is labelled through a three-stage *algorithm-plus-multi-expert* pipeline. *(i)* Multimodal anomaly detectors extract candidate evidence points from raw Metrics, Logs, and Traces; *(ii)* three SRE experts independently confirm or revise the candidates per case without seeing each other's outputs; *(iii)* a senior SRE expert adjudicates remaining disagreements. Every fault scenario is also *re-injected multiple times* before admission, so published labels reflect what a domain expert [1, 11] *should observe* in the data, not what the injection command did.

Each case's ground truth (Fig. 4) pairs the fault metadata—a (category, type, instance) triple plus localization fields whose semantics depend on the (instance type, fault category) combination, since a network attack centers on a source–destination service pair while a node fault centers on a node—with three reasoning-trace labels: a list of per-modality key observations and a list of must-hit metric names the agent's trace should cover (driving the Explainability score), and a list of semantically equivalent phrasings of the fault type (driving the Type Accuracy score, Section 4.4).

4.4 Evaluation Metric

Design rationale. The protocol is organized along the three pillars of Section 1 and weighted 0.40/0.40/0.20: *Localization* and *Identification* (80%) form the actionable conclusion, and *Reason* (20%) is split into two interlocking process dimensions that constrain each other to prevent reward hacking on the reasoning trace.

Table 3: *AIOps2025* statistical properties.

Metric	Value
Injection-level distribution	service 195 / pod 123 / node 82
Cross-modal necessity	$\geq$ 2 modalities 250/400 (62.5%); all 3 124/400 (31%)
Per-modality marginal need	metric 92.8%, log 56.2%, trace 43.0%
Total evidence entries	1,878 (mean 4.7 per case); modality split metric 50.8%, log 40.0%, trace 9.2%

Localization via Location Accuracy (LA). LA is the fraction of cases whose predicted component string-matches the ground truth exactly:

$$\mathrm{LA} = \frac{L_c}{L_t}. \tag{1}$$

Strict matching distinguishes "right service" from "right instance" (e.g., *emailservice* ≠ *emailservice-0*) and prevents fuzzy-matching score inflation. Two fault families need targeted relaxations: for network attacks, LA accepts a hit on either source or destination since evidence is prominent at both endpoints; for pod-level faults, LA requires the exact pod identifier rather than just the service.

Identification via Type Accuracy (TA). TA scores whether the agent's reason field faithfully reflects the true fault type via two-step matching: keyword detection against the ground-truth fault descriptions (any hit yields full credit), with embedding-similarity fallback for unhit cases. The reason field is truncated to its first 20 words to suppress keyword stuffing.

Reason via Explainability and Efficiency. The Reason pillar is captured by two complementary process metrics. The core metric is **Explainability** (0.10), measuring evidence-point coverage:

$$\text{Explainability} = \frac{E_m}{E_t}, \tag{2}$$

where E_t is the total expected evidence points (the union of the GT's key metrics and key observations) and E_m the count hit in any of the agent's trace observations under strict metric-name, log-keyword, or trace-node matching; each observation contributes only its first 20 characters, again to suppress keyword stuffing. **Efficiency** (0.10) counter-balances Explainability by penalizing overly long traces on LA-correct cases:

$$\text{Efficiency} = \min\Big(1.0,\ \exp\big(-\tfrac{APL-5}{5}\big)\Big), \tag{3}$$

where *APL* is the mean *reasoning_trace* length over LA-correct cases. Restricting to LA-correct prevents wrong-answer short-path gaming; together, the two metrics reward traces that are evidence-grounded *and* concise.

Final score. The four dimensions aggregate into a 0–100 score:

$$\text{Final}_A = (0.4\,\mathrm{LA} + 0.4\,\mathrm{TA} + 0.1\,\mathrm{Exp.} + 0.1\,\mathrm{Eff.}) \times 100. \tag{4}$$

4.5 Empirical Analysis

Difficulty distribution. Difficulty is shaped by the coupling between fault category and injection level. The 400 cases distribute across the three injection levels as **service** 195 / **pod** 123 / **node** 82 (Table 3). Most categories tie to one "natural" level—node fault at node, network attack at service—while four categories (pod fault,

jvm fault, stress test, and dns fault) span both service and pod, so the three injection levels expose distinct diagnostic patterns rather than identical cases at different scopes.

Cross-modal reasoning necessity. Co-occurrence signatures over the per-case key observations show that 62.5% **of cases require ≥2 modalities and** 31% **require all three** (Table 3); per-modality marginal necessity is graded (metric 92.8%, log 56.2%, trace 43.0%), providing data-layer evidence for the multimodal coverage principle of Section 3.1.

Large-scale real-world validation. The 2025 CCF AIOps Challenge—the first edition of this long-running venue to move into the LLM-agent regime—uses *AIOps2025* under the four-dimensional protocol of Section 4.4. It attracted 561 **teams and** 1,068 **contestants** from leading universities and industry, demonstrating the protocol's tractability for end-to-end agent submissions at production scale.

5 RCA100: 2025 Alibaba Tianchi AIOps Dataset

RCA100 is built for the Tianchi 2025 AIOps Track (Section 5.5) and instantiates causal-chain coverage. It comprises 103 **fault events** injected via chaos drills on the OpenTelemetry Demo Store deployed on Alibaba Cloud ACK, with **six modalities**—Metrics, Logs, Traces, Events, Alerts, and Topology—totalling ≈ 3.4 GB and released under CC BY-NC-SA 4.0.

5.1 System and Fault Injection

System. The underlying system is the open-source **OpenTelemetry Demo Store** [14], a polyglot e-commerce application of roughly a dozen microservices (Java, C++, Rust, Python) deployed on an **Alibaba Cloud ACK** cluster with managed RDS, Redis, and message-bus backends (Fig. 5). Telemetry is collected through a hybrid **OpenTelemetry + ARMS Agent** pipeline, with the ARMS Agent surfacing JVM-internal state and node-level operational signals not captured by stock OTel. **Aliyun UModel** provides the unified entity-type system that resolves the same logical entity—named differently in APM, K8s, and cloud-resource views—into a single entity ID, enabling chain-step comparability across domains.

Fault injection. Faults are injected via **Chaos Drills**, producing 103 events that span 28 root-cause types aggregated into six semantic groups (Table 4). Each case starts from a single alert event as the diagnostic entry point: 90/103 cases carry a single alert entity (typically at the apm.operation level), while the remaining 13 are kept as composite scenarios with no alert entity to avoid biasing the benchmark toward cases that start from a single entity. Answers are distributed only via an independent answer-key package, never exposed in the public task contract, ensuring blind-evaluation fairness.

5.2 Multimodal Data

Each task ships as a self-contained diagnostic slice of 7 files: 5 Parquets (one each for metrics, logs, traces, events, and alerts) plus the agent-facing task contract and an entity-relation topology snapshot. Metrics use a long-format entity-aligned schema in which every row carries an entity ID resolving to a UModel topology entity;

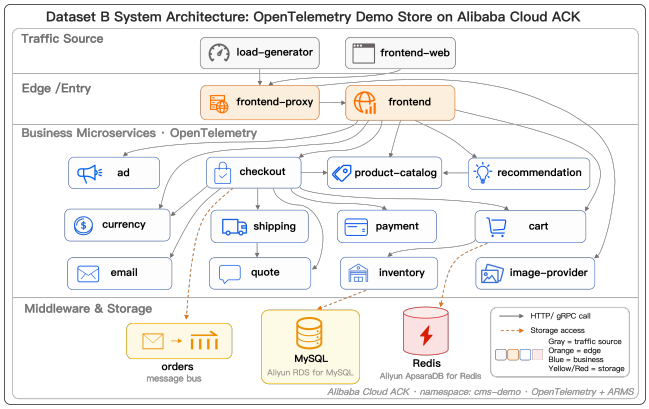

Figure 5: *RCA100* system architecture. The OpenTelemetry Demo Store on Alibaba Cloud ACK: a polyglot microservice e-commerce application entered through *frontend* → *frontend-proxy*, backed by Aliyun RDS for MySQL, ApsaraDB for Redis, and an order message bus, with traces collected through a hybrid OpenTelemetry + ARMS Agent pipeline.

Table 4: *RCA100* fault taxonomy: 28 root-cause types aggregated into 6 semantic groups, with per-type case counts.

Group	Root-cause type	#Cases
Application logic (38)	httpError5xx	11
	rateLimiting	11
	trafficSurge	6
	nullPointerException	4
	trafficHotspot	3
	loadBalancerFailure	2
	codeDefect	1
JVM runtime (16)	memoryPressure	12
	threadExhaustion	2
	fullGC	2
Cloud resource (14)	nodeCpuHigh	12
	nodeDown	1
	nodeMemoryOOM	1
Middleware & DB (13)	slowSQL	5
	redisUnavailable	4
	dbNetworkLatency	2
	messageQueueBacklog	1
	cacheBreakdown	1
K8s lifecycle (12)	replicaScaleDown	6
	resourceLimitMisconfig	1
	podCrashLoop	1
	podPendingUnschedulable	1
	podRestartFlapping	1
	networkPolicyIsolation	1
	dnsResolutionFailure	1
Resource & perf. (10)	cpuFullLoad	8
	cpuDeadLoop	1
	diskIOHigh	1
Total	28 **types** (6 **groups**)	103

Logs and Traces follow the SLS application-log and OpenTelemetry span schemas. Modality samples are in Fig. 2 (Section 3.1).

Higher-order modalities. The three signals beyond the base $M + L + T$ stack play distinct semantic roles. *Events* stream K8s lifecycle

signals (pod restart, scheduling failure, backoff) that are visible only in this modality and indispensable for pod- and node-layer faults. *Alerts* provide the entry-alert lifecycle (∼30 events per case, covering trigger, escalation, and recovery). *Topology* is a per-task UModel entity-relation snapshot at the alert moment, serving as the graph skeleton for cross-modal reasoning.

Scale and integrity. In total *RCA100* comprises 721 **files,** ≈ 116 **M rows,** 3.4 **GB**. We verified full-corpus reference integrity: every cross-modality reference resolves into the corresponding task's topology at 100% with no dangling edges, and GT root-cause entities match the topology at 98.06% (101/103).

5.3 Groundtruth Labeling

The four-layer labels are produced under the same algorithm-plus-multi-expert pipeline as *AIOps2025* (Section 4.3): multimodal detectors propose candidate chain steps and observability checkpoints, three SRE experts independently revise them per case, and a senior expert adjudicates disagreements. Every chaos drill is also re-injected multiple times before admission.

Each case's ground-truth file (Fig. 6) carries four layers: an expected fault type drawn from the 28-class taxonomy, a target-entity list pinning the root cause to UModel entity IDs, a reasoning-step array encoding the causal chain as cause → propagation → impact (91 three-step and 12 four-step chains), and per-step observability checkpoints. The 661 checkpoints (≈ 6.42 per case) span metric, trace, event, and alert sources, and 99.5% carry a ⟨comparator, value, unit⟩ numeric constraint, so the agent must *match the numeric condition*, not merely mention the metric. Common signals include request count (204 checkpoints), average request latency (178), and error count (97).

5.4 Evaluation Metric

Design rationale. The protocol is organized along the three pillars of Section 1 and weighted 0.40/0.30/0.30: Entity Localization leads (0.40) since the located entity is the actuation point of any repair; Fault Identification and Reasoning Process get equal 0.30, since naming the right fault type and walking the right chain contribute symmetrically to diagnostic credibility. The first two dimensions (70%) are quantified deterministically via UModel topology distance, with no LLM-as-judge component in the bulk of the protocol.

Localization via Entity Localization. A UModel entity-ID match with partial credit on topologically adjacent entities (0.40).

Identification via Fault Identification. A fine-grained 28-class match against the expected fault type (0.30).

Reason via Reasoning Process. The Reason pillar is captured by a process score (0.30) jointly measured by causal-chain node match rate and observability-checkpoint hit rate, so an agent must walk the right chain *and* cite the right evidence at each step.

Final score. The three dimensions aggregate into a 0–100 score:

$$\text{Final}_B = (0.4\,\text{Entity} + 0.3\,\text{Fault} + 0.3\,\text{Process}) \times 100. \quad (5)$$


```
{
  "case_id": "F014-httpError5xx.tbdh9alum56k...",
  "outcome": {
    "expected_fault_id": "F014-httpError5xx",
    "target_entities": [
      {"entity_name": "payment", "entity_type":
         "apm.service"}
    ]
  },
  "reasoning": { "steps": [
    {"step": 1, "step_type": "cause",
     "target": "payment",
     "observability": [
       {"signal": "error_count",
        "expected": {"comparator": ">=", "value": 882
           9}},
       {"signal": "error_rate",
        "expected": {"comparator": ">=", "value": 0
           .50}}
     ]},
    {"step": 2, "step_type": "propagation",
     "target": "checkout",
     "observability": [
       {"signal": "error_count",
        "expected": {"comparator": ">=", "value": 882
           8}},
       {"signal": "error_rate",
        "expected": {"comparator": ">=", "value": 0
           .50}}
     ]},
    {"step": 3, "step_type": "impact",
     "target": "checkout::PlaceOrder",
     "observability": [
       {"signal": "error_count",
        "expected": {"comparator": ">=", "value": 882
           8}}
     ]}
  ]}
}
```


Figure 6: *RCA100* ground-truth example: an httpError5xx case propagating along *payment* → *checkout* → *PlaceOrder*. Each chain step carries a typed role and numeric observability checkpoints the agent must match.

5.5 Empirical Analysis

Difficulty distribution. Difficulty is multidimensional. The 28 root-cause types follow a long-tail distribution: the top four (nodeCpuHigh, memoryPressure, httpError5xx, and rateLimiting) account for 44.7% of cases while the remaining 24 form the long tail. Root causes sit at the APM service level in 83 cases, the K8s node level in 17, and the K8s pod level in 3 (Table 6); the 20/103 (19.4%) cases whose chain crosses the APM ↔ K8s domain boundary form the hardest subset for cross-domain attribution.

Cross-modal reasoning necessity. Although 98.2% of the 661 checkpoints sit on metric, the other modalities carry *structural* rather than counted evidence: 92.2% of chains traverse ≥ 2 entity kinds (relying on Topology's cross-domain alias edges for normalization), and the same 19.4% of cases with K8s-layer root causes

Table 5: Design comparison of *AIOps2025* and *RCA100*.

Dimension	*AIOps2025*	*RCA100*
Core target	Reasoning-process explainability	Causal-chain reasoning correctness
Scale	400 cases	103 cases
System	HipsterShop on self-hosted K8s (~13 services)	OTel Demo Store on Aliyun ACK (~a dozen services)
Modalities	3 (Metrics / Logs / Traces)	6 (+Events / Alerts / Topology)
Data layout	Per day × hour; shared across cases	Per-task self-contained (7 files per task)
Data scale	2,835 files / 269 M rows / 11.9 GB	721 files / 116 M rows / 3.4 GB
Input form	Open NL anomaly description + time window	Structured alert event in NL prompt (90/103 with alert entity)
Topology	Implicit (must be inferred)	Explicit entity graph (per-task snapshot)
Taxonomy	9 categories × 18 types × 3 levels	28 root-cause types → 6 semantic groups
Causal chain	None (single-step localization)	3–4 steps explicit (91 / 12)
Evidence	Per-modality key-evidence list	Anchored to chain steps (checkpoints)
Evaluation	LA / TA / Exp. / Eff. at 0.40/0.40/0.10/0.10	Entity / Fault / Process at 0.40/0.30/0.30

Table 6: *RCA100* statistical properties.

Metric	Value
Root-cause-level distribution	apm.svc 83 / k8s.node 17 / k8s.pod 3
Fault-type concentration	top-4 types 44.7%; remaining 24 form a long tail
Multi-entity chains	95/103 (92.2%) traverse ≥ 2 entity kinds
Cross-domain chains	20/103 (19.4%) cross APM ↔ K8s
Avg. modalities touched	4.02/6 per case
Step-type counts	cause 103 / propagation 115 / impact 103

depend on lifecycle evidence visible only in Events. On average each case touches 4.02 of the 6 modalities.

Large-scale real-world validation. The Tianchi 2025 AIOps Track—a track of Alibaba Cloud's *2025 AI-Native Programming Challenge*—uses *RCA100* under the three-dimensional protocol of Section 5.4. It attracted 5,532 **teams** of cloud-platform practitioners and academic researchers, validating the causal-chain protocol at production scale.

5.6 Comparison of the Two Datasets

Table 5 contrasts the two datasets along 12 dimensions. They share the same essence—multi-layer microservice telemetry evaluated across modalities—and differ in how they expose the reasoning process: *AIOps2025* labels evidence as a flat per-modality list (*key-evidence coverage*), while *RCA100* labels it as a typed causal chain (*causal-chain coverage*). Both raise the explainability floor of agent diagnosis. As supervision signals for agent training the two are complementary: key-evidence labels are cheaper to obtain at scale, while causal-chain labels are more expensive but more informative, since they expose how evidence chains into a propagation explanation. Future microservice RCA datasets should invest in causal-chain labelling whenever the annotation budget allows.

6 Discussion

6.1 Fault Injections ≠ Fault Cases

A common misconception in chaos-driven benchmark construction is to equate "a fault injection" with "a usable fault case." The loss between the two is substantial, in three recurring patterns: *(i) absorption by fault tolerance*—retries, degradation, or autoscaling silently mask the fault; *(ii) faint footprints*—an alert fires but the trace is too thin for even a domain expert to reverse-engineer a complete causal chain; *(iii) divergence from intent*—e.g., CPU pressure unexpectedly triggering an OOM kill, so the GT label and actual behavior fall out of sync. *AIOps2025*'s 400 and *RCA100*'s 103 cases are accordingly drawn from substantially larger injection pools, admitting only those with a complete, expert-traceable observability footprint. *Benchmark difficulty is not the difficulty of injecting a fault, but the difficulty of diagnosing one from its observability footprint.*

6.2 Data Cleaning for Downstream Reuse

Raw observability data is voluminous, inconsistently named, and laced with irregular cross-modal references; released as is, most participant effort would go into preprocessing rather than diagnosis. We therefore cleaned both datasets along three axes: *cross-modal entity alignment* (service/pod names in *AIOps2025*, UModel entity IDs in *RCA100*), *redundant-slice trimming* (per-task self-contained files in *RCA100*; per domain/day/hour partitioning in *AIOps2025*), and *field standardization* (timestamps, nulls, modality identifiers, Parquet types).

This is not merely a benchmark convenience. Production telemetry exposes the same heterogeneity at greater scale—fragmented across time-series databases (Prometheus, InfluxDB), log backends (Elasticsearch), tracing stores, and per-vendor APIs. An agent dropped onto raw production data must adapt to each surface before any reasoning can begin, inflating engineering cost and amplifying hallucination. A more workable direction is to push unification into a dedicated tool layer—e.g., agent-ready observability data modeling [15]—so the agent queries one normalized interface rather than negotiating N backends. Our cleaning mirrors that pattern at benchmark scale.

6.3 Limitations

We acknowledge four limitations. *(i) Scale gap to production.* Both datasets are built on open-source demos of around a dozen services; production systems can run hundreds to thousands across complex middleware and cross-region deployments, and transfer to that scale remains to be verified. *(ii) Boundary of fault realism.* Chaos-injected faults are relatively clean (single root cause, controlled parameters), whereas production failures often present as concurrent causes,

intermittent flares, or long-accumulated degradation. *(iii) Boundary of label coverage.* Even with multi-expert labelling, valid but non-mainstream diagnostic paths can be omitted; agents that reach the correct root cause via an unlabelled trace are under-credited. *(iv) Insufficient coverage of efficiency. AIOps2025*'s Efficiency metric uses reasoning-trace length as a proxy and does not capture end-to-end latency, token consumption, or tool-call cost.

6.4 Beyond RCA: Extended Uses of the Datasets

Multimodal completeness and fine-grained expert labels make both datasets reusable beyond microservice RCA: the time-aligned Metrics+Logs+Traces support multimodal time-series anomaly detection; *RCA100*'s causal-chain labels and entity-relation graphs supply ground truth for causal discovery and graph neural methods; *AIOps2025*'s per-modality key-evidence labels are rare material for studying expert–agent reasoning-path divergence and agent self-evaluation; and the real-failure traces support training fault-synthesis or fault-injection models.

More broadly, the underlying paradigm is not specific to microservices: any task that rests an answer on evidence and admits structural labelling of that evidence—medical diagnosis, legal reasoning, scientific discovery—can adopt the same key-evidence and causal-chain coverage forms, turning "does the agent's reasoning rest on the right evidence" into a computable scoring signal.

7 Conclusion

We presented two complementary microservice AIOps benchmarks that operationalize *reasoning-process evaluation*: 400 HipsterShop cases anchor the diagnosis to per-modality key evidence, and 103 OpenTelemetry Demo Store cases anchor it to typed causal chains over a UModel topology. Both have been stress-tested at scale—6,093 teams across the 2025 CCF AIOps Challenge and the Tianchi AIOps Track—and are released with their scoring code. We hope the datasets shift agentic diagnosis benchmarking from final-answer matching toward evidence-grounded reasoning, in microservice RCA and in adjacent evidence-driven agent tasks.

References

[1] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site reliability engineering: how Google runs production systems.* O'Reilly Media, Inc.

[2] Chaos Mesh Authors. 2020. Chaos Mesh: A Powerful Chaos Engineering Platform on Kubernetes. https://chaos-mesh.org/.

[3] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. Aiopslab: A holistic framework to evaluate ai agents for enabling autonomous clouds. *Proceedings of Machine Learning and Systems* 7 (2025).

[4] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. 2024. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. 674–688.

[5] Jackson Clark, Yiming Su, Saad Mohammad Rafid Pial, Yifang Tian, Lily Gniedziejko, Hans-Arno Jacobsen, Yinfang Chen, and Tianyin Xu. 2026. SREGym: A Live Benchmark for AI SRE Agents with High-Fidelity Failure Scenarios. *arXiv preprint arXiv:2605.07161* (2026).

[6] Google Cloud Platform. 2018. Online Boutique (HipsterShop): A Cloud-First Microservices Demo Application. https://github.com/GoogleCloudPlatform/microservices-demo.

[7] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, et al. 2020. TiDB: a Raft-based HTAP database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3072–3084.

[8] Azam Ikram, Sarthak Chakraborty, Subrata Mitra, Shiv Saini, Saurabh Bagchi, and Murat Kocaoglu. 2022. Root cause analysis of failures in microservices through causal discovery. *Advances in Neural Information Processing Systems* 35 (2022), 31158–31170.

[9] Jaeger Authors. 2017. Jaeger: Open Source, End-to-End Distributed Tracing. https://www.jaegertracing.io/.

[10] Saurabh Jha, Rohan Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, et al. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. *Proceedings of Machine Learning Research* 267 (2025), 27134–27197.

[11] Jiajun Jiang, Weihai Lu, Junjie Chen, Qingwei Lin, Pu Zhao, Yu Kang, Hongyu Zhang, Yingfei Xiong, Feng Gao, Zhangwei Xu, et al. 2020. How to mitigate the incident? an effective troubleshooting guide recommendation technique for online service systems. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1410–1420.

[12] JinJin Lin, Pengfei Chen, and Zibin Zheng. 2018. Microscope: Pinpoint performance issues with causal graphs in micro-service environments. In *International Conference on Service-Oriented Computing*. Springer, 3–20.

[13] Yuhe Liu, Changhua Pei, Longlong Xu, Bohan Chen, Mingze Sun, Zhirui Zhang, Yongqian Sun, Shenglin Zhang, Kun Wang, Haiming Zhang, et al. 2023. Opseval: A comprehensive it operations benchmark suite for large language models. *arXiv preprint arXiv:2310.07637* (2023).

[14] OpenTelemetry Authors. 2022. OpenTelemetry Demo: A Microservice-based Distributed Application. https://github.com/open-telemetry/opentelemetry-demo.

[15] Changhua Pei, Zheyuan Li, Zexin Wang, Hang Cui, Xiaohui Nie, Qi Zhou, Fang Situ, Cheng Zhang, Xin Zhang, Xidao Wen, et al. 2026. UModel: An Agent-Ready Observability Data Modeling Method at Scale. *arXiv preprint arXiv:2606.04799* (2026).

[16] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, et al. 2025. Flow-of-action: SOP enhanced LLM-based multi-agent system for root cause analysis. In *Companion Proceedings of the ACM on Web Conference*. 422–431.

[17] Luan Pham, Huong Ha, and Hongyu Zhang. 2024. BARO: Robust root cause analysis for microservices via multivariate Bayesian online change point detection. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2214–2237.

[18] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. RCAEval: a benchmark for root cause analysis of microservice systems with telemetry data. In *Companion Proceedings of the ACM on Web Conference 2025*. 777–780.

[19] Prometheus Authors. 2012. Prometheus: Monitoring System and Time Series Database. https://prometheus.io.

[20] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, Vol. 2024. 9695–9717.

[21] Yongqian Sun, Yu Luo, Xidao Wen, Yuan Yuan, Xiaohui Nie, Shenglin Zhang, Tong Liu, and Xi Luo. 2025. TrioXpert: An automated incident management framework for microservice system. In *IEEE/ACM 40th International Conference on Automated Software Engineering*. IEEE, 3239–3250.

[22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[23] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. 2020. MicroRCA: Root cause localization of performance issues in microservices. In *IEEE/IFIP Network Operations and Management Symposium*.

[24] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. Openrca: Can large language models locate the root cause of software failures?. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

[26] Shenglin Zhang, Pengxiang Jin, Zihan Lin, Yongqian Sun, Bicheng Zhang, Sibo Xia, Zhengdan Li, Zhenyu Zhong, Minghua Ma, Wa Jin, et al. 2023. Robust failure diagnosis of microservice system through multimodal data. *IEEE Transactions on Services Computing* 16, 6 (2023), 3851–3864.

[27] Shenglin Zhang, Sibo Xia, Wenzhao Fan, Binpeng Shi, Xiao Xiong, Zhenyu Zhong, Minghua Ma, Yongqian Sun, and Dan Pei. 2025. Failure diagnosis in microservice systems: A comprehensive survey and analysis. *ACM Transactions on Software Engineering and Methodology* 35, 1 (2025), 1–55.

[28] Wei Zhang, Hongcheng Guo, Jian Yang, Zhoujin Tian, Yi Zhang, Yan Chaoran, Zhoujun Li, Tongliang Li, Xu Shi, Liangfan Zheng, et al. 2024. mABC: Multi-agent blockchain-inspired collaboration for root cause analysis in micro-services architecture. In *Findings of the Association for Computational Linguistics: EMNLP*. 4017–4033.

[29] Xuanhe Zhou, Guoliang Li, Zhaoyan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. 2024. D-Bot: Database Diagnosis System using Large Language Models. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2514–2527.